WEEK 04

Functions and returned values

Python foundations

Functions and returned values

Function contracts, local names and returned data

Define and call

Parameters name inputs; arguments supply values for one call.

```
def double(n): return n * 2
print(double(3))  # 6
```

Return a reusable value

return sends data to the caller; print displays it.

```
def double(n): return n * 2
answer = double(3) + 1  # 7
```

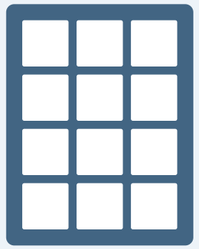
Local names start fresh

Each call has its own parameter and local bindings.

```
def increment(n): return n + 1
print(increment(2), increment(2))  # 3 3
```

The same rule measures different mats

3 × 4 floor mat



12 square units

area(width, height)

return width * height

Different inputs; same rule.

2 × 5 floor mat



10 square units

```
def area(width, height):
    return width * height
print(area(3, 4), area(2, 5))
```

return gives the caller a value it can store or reuse.

Syntax to read and reuse

Python / syntax	Meaning	When to use it
def name(argument):	Define a callable block	Reuse one rule
return result	Send a value to the caller	Make the result testable
name(value)	Call with an argument	Run the function body
from module import name	Reuse a definition	Share logic across files

Python Basics · sections 07, 08, 10, 15 · use only when you need a lookup.

Define first, then call

```
01 def double(number):
02     return number * 2
03
04 answer = double(4)
05 print(answer)
06 assert answer == 8
```

1. def stores the function instructions.

A returned value can be reused

```
01 def add_one(number):
02     result = number + 1
03     return result
04
05 first = add_one(4)
06 second = add_one(8)
07 print(first, second, first + second)
```

1. The caller receives a value it can store, add or test.

Return one dictionary with named fields

```
01 def make_score(name, points):
02     return {"name": name, "points": points}
03
04 note = make_score("Ada", 4)
05 print(note["name"])
06 print(note["points"])
```

1. A function can return the same dictionary shape you learned in Week 3.

Define once, call with concrete inputs

```
01 def rectangle_area(width, height):
02     return width * height
03
04 print("defined")
05 result = rectangle_area(3, 4)
06 print(result)
```

1. Expected: defined, then 12.
2. 3 and 4 become this call's local inputs.
3. The caller stores the returned result.

Function definition and function call

```
01 def planned_portions(unit, containers):
02     return unit * containers
03
04 print("Function defined")
05 result = planned_portions(3, 4)
06 print("Planned portions:", result)
07 assert result == 12
```

1. Parameters receive the arguments.
2. return sends the result to the caller.
3. Store, print or test the returned value.

def creates a function; a call executes its body.

return and print have different roles

return

Sends a value to the caller.
The caller can store or test it.

print

Displays text for a reader.
Printing alone does not return that value.

A return value can be reused by another caller.

Optional: main() and the import guard

```
01 def planned_portions(unit, containers):
02     return unit * containers
03
04 def main():
05     print("Planned portions:", planned_portions(3, 4))
06
07 if __name__ == "__main__":
08     main()
```

1. def alone does not call main().
2. Direct execution makes the guard true.
3. Import leaves the guarded call inactive.

main is an ordinary function called by an explicit guard.

Optional: Import and call a function

```
01 from pickup_app import planned_portions
02
03 print("Import completed")
04 print(planned_portions(3, 2))
05 assert planned_portions(3, 2) == 6
```

1. The import has no automatic demo output.
2. An explicit call returns a value.
3. Use the documented working directory.

Import definitions, then call them explicitly.

Debug a function with a boundary case

```
def pickup_label(total_portions, reviewed):
    if not reviewed:
        return "hold"
    if total_portions > 20:  # Deliberate bug.
        return "team"
    return "standard"

print(pickup_label(20, True))
# Observed: standard. Required: team.
```

1. Trace 20 > 20 by hand
2. Keep the agreed expectation unchanged
3. Repair the comparison in your lab and retest

The observed output is evidence of a bug, not the expected policy.

Optional rehearsal: a different boundary

Element	Original	Practice variant
Reviewed team boundary	At least 20	At least 16
Unreviewed request	hold	hold
Meaning	Coordination workload	Coordination workload
Evidence	Boundary repair	Changed and preserved cases

Optional practice only. The Midterm uses the canonical boundary of 20.

Hands-on practice

Guided integration

Python practice

P1–P2 · Core

Two small programs with approach hints.

P3 · Core

A complete program with normal and edge-case tests.

P4–P5 · Optional

An application and a harder challenge.

P1–P3: your code, expected results and an explanation. P4–P5 are optional.

Individual Midterm

Continue implementation → test and package → hand in

The project need

1

Need

The same calculation should work with different inputs.

2

Operation

Write the required summary function, check the four common cases and package it.

3

Result

Another person can run the returned summary with the supplied tests.

The same returned value can serve a notebook cell or an optional supplied form.

Follow one value across the program boundary

1

Inputs

Declared unit, container count and supplied review decision.

2

Calculation

Multiply to obtain planned portions.

3

Label

Apply review priority and the workload threshold.

4

Display

Explain the returned result without claiming a completed pickup.

Keep the unit and authority of every value visible.

Midterm: implement, test, hand in

Supplied starting point

Starter, four common cases and packaging instructions are supplied. No previous notebook must be finished.

You run / explain

Write summarize_request with a loop and return dictionary; add test_midterm.py and README.md; package these with midterm.py.

Check the evidence

Four cases: A hold/20, B standard/9, C team/20, D team/22 with 6 containers. Verify all three returned fields.

Implement, test, package, review and hand in.

Midterm: check the returned values and package

Step	What to show
1 · Locate	Identify the input, this component and its next user.
2 · Implement	Return all three fields; package code, tests and README.
3 · Check	A: hold/20; B: standard/9; C: team/20; D: team/22.
4 · Explain	Show your code, its actual result and why it behaves that way.

Research 3 is assigned after the Midterm; share it in Week 5.

Further reading in English

1. ## Python functions and control flow

   https://docs.python.org/3/tutorial/controlflow.html

2. ## Python main and module execution

   https://docs.python.org/3/library/__main__.html

Read one relevant section, then bring an example and a question.